

CSCI2750 Homework 4

Wong Cheuk Yin 1155192671

April 18, 2026

1 Problem 1

Given the following dataset:

i	$x^{(i)}$	label
1	$(-3, 2, 0)$	+
2	$(-2, -2, 1)$	-
3	$(-1, -1, -1)$	-
4	$(0, 0, 1)$	-
5	$(1, -2, -1)$	-
6	$(2, 1, 2)$	+
7	$(3, -1, 0)$	+

We use the GINI index to determine the best split point.

At the root (7 points: 3 positive, 4 negative), checking candidate threshold splits gives a minimum weighted Gini of $\frac{8}{35} \approx 0.2286$, achieved by $x_1 \leq 1$ and tied with a split on $x_2 \leq 0$. We choose

$$x_1 \leq 1.$$

Then:

- Right child ($x_1 > 1$): $\{x^{(6)}, x^{(7)}\}$, both +, hence pure leaf (+).
- Left child ($x_1 \leq 1$): $\{x^{(1)}, x^{(2)}, x^{(3)}, x^{(4)}, x^{(5)}\}$ with labels (+, -, -, -, -).

Split the left child by

$$x_1 \leq -3,$$

so that:

- Left leaf: $\{x^{(1)}\}$, pure +.
- Right leaf: $\{x^{(2)}, x^{(3)}, x^{(4)}, x^{(5)}\}$, pure -.

Therefore, a 5-node decision tree is:

- If $x_1 > 1$, predict +;
- else if $x_1 \leq -3$, predict +;
- else predict -.

This tree has 2 internal nodes and 3 leaves (total 5 nodes), and it classifies all training points correctly.

$$\text{Training error} = \frac{0}{7} = 0.$$

2 Problem 2

Given the following dataset:

$$\mathbf{a} = (0, 1, 2, 1), \quad \mathbf{b} = (2, 3, 4, 2), \quad \mathbf{c} = (-2, -1, 1, 3), \quad \mathbf{d} = (1, 0, 1, 0), \quad \mathbf{e} = (3, 4, 5, 3).$$

Initially, we have 5 clusters: $C_1 = \{a\}, C_2 = \{b\}, C_3 = \{c\}, C_4 = \{d\}, C_5 = \{e\}$.

Pairwise Manhattan distances:

$$\begin{aligned} d(a, b) &= 7, \quad d(a, c) = 7, \quad d(a, d) = 4, \quad d(a, e) = 11, \\ d(b, c) &= 12, \quad d(b, d) = 9, \quad d(b, e) = 4, \\ d(c, d) &= 7, \quad d(c, e) = 14, \quad d(d, e) = 13. \end{aligned}$$

Hence, the dissimilarity matrix (in the order C_1, C_2, C_3, C_4, C_5) is

$$D = \begin{bmatrix} 0 & 7 & 7 & 4 & 11 \\ 7 & 0 & 12 & 9 & 4 \\ 7 & 12 & 0 & 7 & 14 \\ 4 & 9 & 7 & 0 & 13 \\ 11 & 4 & 14 & 13 & 0 \end{bmatrix}.$$

Observe that minimum Manhattan distance in the matrix is $d(a, d) = 4$ and $d(b, e) = 4$,

First merge: C_4 into C_1

Second merge: C_5 into C_2 .

New clusters: $C_1 = \{a, d\}, C_2 = \{b, e\}, C_3 = \{c\}$.

New dissimilarity matrix (in the order C_1, C_2, C_3):

$$D = \begin{bmatrix} 0 & 7 & 7 \\ 7 & 0 & 12 \\ 7 & 12 & 0 \end{bmatrix}.$$

Observe that minimum Manhattan distance in the matrix is $d(C_1, C_3) = 7$ and $d(C_1, C_2) = 7$.

Third merge(arbitrarily chosen since tied): C_3 into C_1

New clusters: $C_1 = \{a, c, d\}, C_2 = \{b, e\}$. (The result clusters for $K=2$)

Then we merge C_1 and C_2 into C_1 (because dendrogram is a tree, which means $K=1$) and get the final clusters: $C_1 = \{a, c, d, b, e\}$.

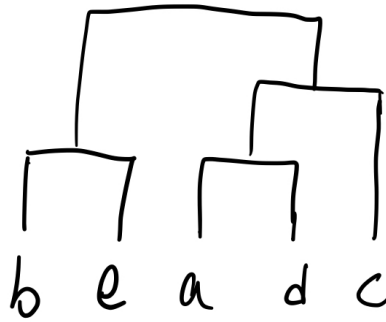


Figure 1: Resultant Dendrogram.

3 Problem 3

Given

$$\mathcal{D} = \{x^{(i)} \mid x^{(i)} = 2i, i = 1, \dots, m\}.$$

(a) Case $m = 6$, initial center $z_1 = x^{(1)}$, and $K = 3$

Data points:

$$\mathcal{D} = \{2, 4, 8, 16, 32, 64\}.$$

K-centers iterations:

- Given initial center: $z_1 = 2$
- Step 1: choose $z_2 = 64$ (farthest point from z_1).
- Step 2: choose $z_3 = 32$ (farthest point from z_1 and z_2).

Final centers:

$$z_1 = 2, \quad z_2 = 64, \quad z_3 = 32.$$

Resultant clusters (by nearest-center assignment):

$$Z_1 = \{2, 4, 8, 16\}, \quad Z_2 = \{64\}, \quad Z_3 = \{32\}.$$

Radius of clustering:

$$\mathcal{R}(Z) = \max_{x \in \mathcal{D}} \min_{k \in \{1, 2\}} |x - z_k| = 16 - 2 = 14.$$

(b) Compute SSE and BSS for the clustering result in (a)

Mean of each cluster:

$$\mu_1 = \frac{2 + 4 + 8 + 16}{4} = 7.5, \quad \mu_2 = 64, \quad \mu_3 = 32.$$

SSE:

$$\sum_{k=1}^3 \sum_{x \in Z_k} (x - \mu_k)^2 = ((2 - 7.5)^2 + (4 - 7.5)^2 + (8 - 7.5)^2 + (16 - 7.5)^2) + (64 - 64)^2 + (32 - 32)^2 = 115$$

Global mean:

$$\bar{x} = \frac{2 + 4 + 8 + 16 + 64 + 32}{6} = 16.$$

BSS:

$$\sum_{k=1}^3 |Z_k| (\mu_k - \bar{x})^2 = 4 \cdot (7.5 - 16)^2 + 1 \cdot (64 - 16)^2 + 1 \cdot (32 - 16)^2$$

(c) Proof for $\mathcal{R}(Z) \leq 2^{m-1}$ for $K=2$

With no fixed initial center, z_1 will be chosen randomly from the dataset.

If $z_1 = 2^i$ for some $i \in \{1, 2, \dots, m-1\}$, z_2 will always be 2^m since it must be the farthest point from z_1 . In this case, all remaining points will be assigned to Z_1 because they are all closer to z_1 than to z_2 . Hence, we have:

$$Z_1 = \{2, 4, 8, \dots, 2^{m-1}\}, \quad Z_2 = \{2^m\},$$

To get the greatest possible radius, z_1 should be 2 or 2^{m-1} since the distance between 2 and 2^{m-1} is the greatest.

$$\mathcal{R}(Z) = \max_{x \in \mathcal{D}} \min_{k \in \{1, 2\}} |x - z_k| = 2^m - 2^{m-1} = 2^{m-1}.$$

Case 2, If $z_1 = 2^m$, z_2 will always be 2 since it must be the farthest point from z_1 . In this case, all remaining points will be assigned to Z_2 because they are all closer to z_2 than to z_1 . Hence, we have:

$$Z_1 = \{2^m\}, \quad Z_2 = \{2, 4, 8, \dots, 2^{m-1}\},$$

To get the greatest possible radius, z_1 should be 2^m since the distance between 2^m and 2 is the greatest.

$$\mathcal{R}(Z) = \max_{x \in \mathcal{D}} \min_{k \in \{1, 2\}} |x - z_k| = 2^m - 2 = 2^{m-1}.$$

We can see, there is no case that can get a radius greater than 2^{m-1} , Therefore,

$$\mathcal{R}(Z) \leq 2^{m-1}.$$

4 Problem 4

4.1 (a) Set of unique items

$$\mathcal{I} = \{\text{Phone Charger, Passport, Water Bottle, Power Bank}\}.$$

4.2 (b) Apriori algorithm

There are 7 transactions, so

$$\text{supp}(X) = \frac{\#\{\text{transactions containing } X\}}{7}.$$

With minsup = 0.7, an itemset must appear at least 5 times to be frequent.

All itemsets and supports

Itemset X	$\text{supp}(X)$	Frequent?
{Phone Charger}	6/7	Yes
{Passport}	4/7	No
{Water Bottle}	6/7	Yes
{Power Bank}	3/7	No
{Phone Charger, Passport}	3/7	No
{Phone Charger, Water Bottle}	5/7	Yes
{Phone Charger, Power Bank}	3/7	No
{Passport, Water Bottle}	3/7	No
{Passport, Power Bank}	1/7	No
{Water Bottle, Power Bank}	3/7	No
{Phone Charger, Passport, Water Bottle}	2/7	No
{Phone Charger, Passport, Power Bank}	1/7	No
{Phone Charger, Water Bottle, Power Bank}	3/7	No
{Passport, Water Bottle, Power Bank}	1/7	No
{Phone Charger, Passport, Water Bottle, Power Bank}	1/7	No

Therefore, the frequent itemsets (Apriori output) are

$$\mathcal{I}^{(1)} = \{\{\text{Phone Charger}\}, \{\text{Water Bottle}\}\}, \quad \mathcal{I}^{(2)} = \{\{\text{Phone Charger, Water Bottle}\}\}$$

4.3 (c) Strong association rules

Since $\text{minconf} = 0.8$, the strong rules from $\{\text{Phone Charger, Water Bottle}\}$ are:

$$\{\text{Phone Charger}\} \Rightarrow \{\text{Water Bottle}\}, \quad \text{conf} = \frac{\text{supp}(\{\text{Phone Charger, Water Bottle}\})}{\text{supp}(\{\text{Phone Charger}\})} = \frac{5/7}{6/7} = \frac{5}{6} \approx 0.833.$$

$$\{\text{Water Bottle}\} \Rightarrow \{\text{Phone Charger}\}, \quad \text{conf} = \frac{\text{supp}(\{\text{Phone Charger, Water Bottle}\})}{\text{supp}(\{\text{Water Bottle}\})} = \frac{5/7}{6/7} = \frac{5}{6} \approx 0.833.$$

4.4 (d) Recommendation for Aaron

Recommend **Water Bottle**. Aaron already has Phone Charger; the strong rule $\{\text{Phone Charger}\} \Rightarrow \{\text{Water Bottle}\}$ is a strong association rule, so among people who pack a phone charger, a water bottle is a highly associated next item, and Aaron did not list it.

4.5 (e) Recommendation for Hannah

Recommend **Phone Charger**. Hannah already has Water Bottle; the strong rule $\{\text{Water Bottle}\} \Rightarrow \{\text{Phone Charger}\}$ is a strong association rule, so among people who pack a water bottle, a phone charger is a highly associated next item, and Hannah did not list it.

5 Problem 5

5.1 (a) Study of MATLAB SVD implementation

For image preprocessing, the given code did:

- Convert the image to grayscale (`rgb2gray`).
- Cast pixel values to `double` because the build-in `svd` function requires double.
- Normalize intensities by dividing by $\max(X(:))$, so entries lie in $[0, 1]$.

Then, function `svd()` was called to compute the corresponding U, S, V , which comes as single vector. To get the full matrices, we need to reshape the vectors back into the original dimensions by the following code:

```
U_new = U(:, 1:k);
S_new = diag(S);
S_new = diag(S_new(1:k));
V_new = V(:, 1:k);
```

- U is duplicated to every row of the matrix to get $\bar{U}$.
- Let the i -th diagonal element of $\bar{S}$, the i -th element of the S . Other elements are set to 0.
- V is duplicated to every column of the matrix to get $\bar{V}$.

Finally, we can reconstruct the image by $X = \bar{U}\bar{S}\bar{V}'$ which is implemented in the following code:

```
X_new = U_new * S_new * V_new';
```

5.2 (b) SVD compression on a chosen image

Chosen image:



Figure 2: An evil orange cat(right) attempting to execute a Siamese cat(left) by shutting down his respiratory system forever. I hope he survives. :(

The image's resolution is 1080×1080 , which means:

$$m = 1080, \quad n = 1080.$$

We need to follow the rule below to make the compression meaningful:

$$k < \frac{mn}{m+n+1} = \frac{1080 \times 1080}{1080 + 1080 + 1} \approx 539.75.$$

we choose $k = 200$ and compute the the compressed image with MATLAB code implemented with the logic in (a).

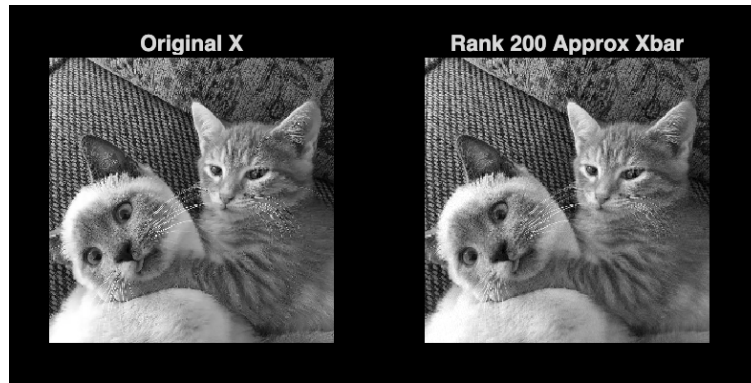


Figure 3: Visualisation of X and its rank-200 approximation $\bar{X}$.

Apparently, the quality did not drops significantly. The cat is still recognizable.

5.3 (c) Compression with different compression rates

We can obtain k needed to achieve the desired compression rate by the following formula:

$$\text{Compression rate} = \frac{\text{storage}(\bar{X})}{\text{storage}(X)} = \frac{k(m+n+1)}{mn} = \frac{k \times (1080 + 1080 + 1)}{1080 \times 1080}$$

$$k = \frac{1080^2}{2161} \times \text{Compression rate}$$

We can obtain the following table:

Compression rate	Required rank k (closest integer)
10%	54
20%	108
40%	216
80%	432

Visualisation by MATLAB code:

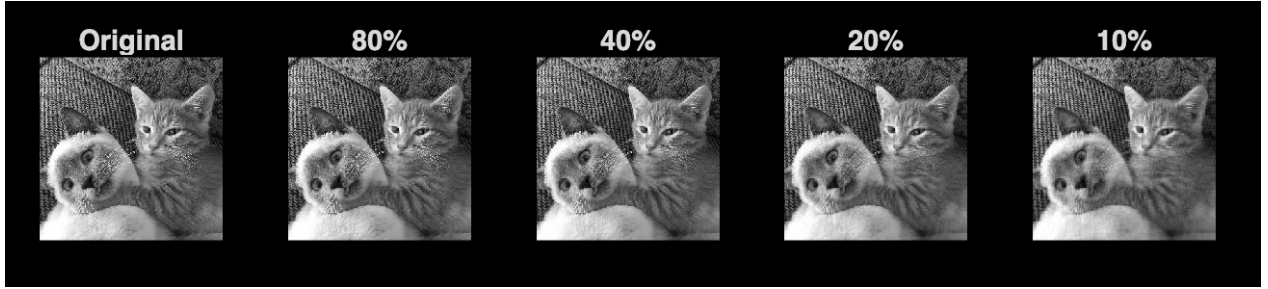


Figure 4: Visualisation of the compressed images with different compression rates.

5.4 (d) SVD for RGB image

Skip the $X = \text{rgb2gray}$ in order to keep the original RGB image. We can compute the SVD with the following code:

```
k = 200; %change if needed
X = imread('orangecat.jpeg');
X = double(X);
X = X ./ max(X(:));

R = X(:, :, 1);
G = X(:, :, 2);
B = X(:, :, 3);
[Ur, Sr, Vr] = svd(R);
[Ug, Sg, Vg] = svd(G);
[Ub, Sb, Vb] = svd(B);

Rbar = Ur(:,1:k) * Sr(1:k,1:k) * Vr(:,1:k)';
Gbar = Ug(:,1:k) * Sg(1:k,1:k) * Vg(:,1:k)';
Bbar = Ub(:,1:k) * Sb(1:k,1:k) * Vb(:,1:k)';

Xbar = cat(3, Rbar, Gbar, Bbar);
```

```
figure;  
subplot(1, 2, 1);  
imshow(X);  
title('Original RGB');  
subplot(1, 2, 2);  
imshow(Xbar);  
title(sprintf('RGB per-channel rank-%d SVD', k));
```

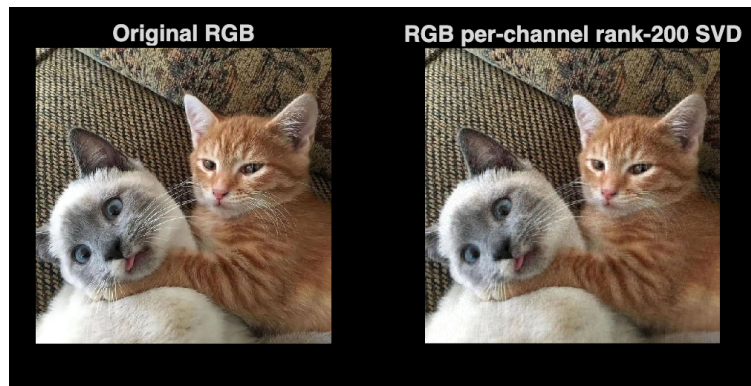


Figure 5: Visualisation of the original RGB image and its rank-200 approximation.

The quality is still good. The cat is still recognizable.